Batch 1210 Mass-Failure — Root-Cause Analysis

Question: Since the framework was running smoothly (only a small number of failures), which change caused batch 1210 to finish with 48 of 50 companies failed?

Answer (proven): The mass failures are **timeouts caused by a 3–8× explosion in per-company discovery workload**, introduced by two commits deployed *after* the last smooth run — `42e7ddf` and `fe26685`. They are **not** corpus/quality failures and are **not** caused by the domain-filter / issuer-token changes in `aba322c`.

---

1. The two reference points

Batch	When	Commit	Result	Meaning
1201	Oct 6 09:36	`43f35ed`	50 completed, **0 failed**	"running smoothly" baseline
1210	Oct 6 23:30	`bb8e08b`	2 completed, **48 failed**	the regression

`git log 43f35ed..HEAD` — everything that shipped between the two runs:

```
bb8e08b  fix(worker): stop Worker-stacking restart loop       <- worker.ts only (pri-
or fix)
aba322c  fix(discovery): reject non-issuer aggregator domains; env-tunable search
timeout
fe26685  feat(discovery+pipeline): issuer-domain PDF sweep + post-fetch aboutness
401ed91  docs: draft proposal A                                <- docs only, no code
42e7ddf  discovery: variants=4, deep num, analyst PDF query, known-doc URL seeding
```

The baseline `43f35ed` has **zero** references to `DISCOVERY_DEEP_NUM` (`git show 43f35ed:server/lib/discovery.ts | grep -c DISCOVERY_DEEP_NUM` → `0`), confirming the expansion below did not exist during the smooth run.

---

2. What the failures actually are (DB ground truth, batch 1210)

Grouped `last_error` of the 48 failed jobs:

Failure reason	Count	What it means
`Discovery timeout: <co> exceeded 1200s`	**~33**	discovery phase itself ran past the 20-min ceiling
`Pipeline timed out after ~2100–2300s`	~10	the whole-job watchdog fired downstream of slow discovery
DB connection timeouts	~5	PG pool (max 10) starved while many jobs sat in slow discovery
`Job watchdog timeout after 2400000ms`	2	40-min hard job cap
`Insufficient corpus (primaries missing)`	**1**	the *only* genuine quality/corpus failure

This split is the decisive evidence. If the `aba322c` domain filters (issuer-token gate, expanded `EXCLUDED_DOMAINS`) were starving the corpus, there would be *many* "Insufficient corpus" failures. There is exactly **one**. The other **47 are latency failures**. Discovery got *slower*, not *stricter*.

---

3. The discovery timeout did not change

`server/lib/discovery.ts:3252`

```
const DISCOVERY_TIMEOUT_MS = Math.max(
  60_000,
  parseInt(process.env.DISCOVERY_TIMEOUT_MS || String(20 * 60 * 1000), 10) || 20 * 60
* 1000,
);
```

This 20-minute (1200s) budget was set on **Oct 2** by commit `977e0ac` — *before both* the smooth run and the failing run. The wall didn't move; the work per company grew until it hit the wall.

---

4. The culprit: per-company discovery workload exploded (code-proven)

4a. `42e7ddf` — "variants=4, deep num, analyst PDF, known-doc seeding"

(i) Results-per-query tripled on the two highest-recall lanes.
Pipeline passes `searchDepth = settings.search_depth || "10"` → **10**

(`server/lib/pipeline.ts:239`). Before `42e7ddf` , Lane 2 (domain) and Lane 6 (variant) used `num: searchDepth` = **10**. After:

```
// server/lib/discovery.ts
const DISCOVERY_DEEP_NUM = parseInt(process.env.DISCOVERY_DEEP_NUM || "30", 10);   //
=30
...
const results = await webSearch(domainQueries[qi], { num: DISCOVERY_DEEP_NUM });   //
Lane 2: 10 -> 30
const results = await webSearch(query,            { num: DISCOVERY_DEEP_NUM });   //
Lane 6: 10 -> 30
```

→ **3× more candidates per query** on the two lanes that run the most queries (Lane 2 runs up to `MAX_LANE2_QUERIES` = 60).

(ii) Query variants 3 → 4. `discovery_query_variants` default raised 3→4 (`pipeline.ts:240`), *and* a one-shot migration bumped existing workspaces still at 3 up to 4. Combined with (i): Lane 6 went from `3 × 10 = 30` to `4 × 30 = 120` candidates — **4×**.

(iii) +1 extra domain query per domain (`buildDomainQueries` , spliced at the front): `site:<domain> (analyst report OR presentation OR factbook OR "fact book") filetype:pdf` , each returning up to 30.

(iv) Known-doc URL seeding re-issues up to `KNOWN_DOC_URLS_MAX` = **40** stored issuer URLs as live candidates every run (default ON) — fetched + graded live. These accumulate across runs (logs show “persisted 40 issuer URLs”).

4b. `fe26685` — “issuer-domain PDF sweep + post-fetch aboutness” (default ON)

(i) Issuer PDF sweep: +1 `site:<domain> filetype:pdf` search *per domain in the family* (num=30), admitting up to `ISSUER_SWEEP_MAX` = **25** PDFs per domain as live candidates (lane `domain-sweep`). For a 2–3 domain family that is +50–75 extra PDFs to fetch and grade.

(ii) Post-fetch aboutness scoring (`ABOUTNESS_ENABLED` default true) runs `scoreAboutness()` on **every fetched document** at three points (fetch loop, reuse-verify, corpus assembly), and corpus assembly additionally calls `storage.getCompanies(workspaceId)` once per company to build the peer-name list. Added CPU per doc on top of far more docs.

4c. `aba322c` — secondary / not the cause

Adds the issuer-token overlap gate to `discoverCrossBrandSiblings` , expands `EXCLUDED_DOMAINS` , and makes the search timeout env-tunable (`SEARCH_TIMEOUT_MS` , default 8000, down from 15000). These are *filters* (they reduce candidates) — consistent with the single corpus failure, not the 47 timeouts. The shorter search timeout only adds latency if searches retry on timeout; it is at most a minor contributor.

---

5. Empirical confirmation (live worker logs)

Discovery is now emitting **enormous candidate counts per company** (worker logs, Oct 7):

```
[Aura Minerals Inc.]  Discovery found 421 total candidates
[PPG INDUSTRIES, INC.] Discovery found 858 total candidates
[PPG INDUSTRIES, INC.] Discovery found 828 total candidates
[*] known-doc-seed: persisted 40 issuer URLs (27 from this run)
```

400–860 candidates/company to search for, resolve, rank (and then fetch) is exactly what blows the 1200s discovery budget. At the smooth baseline (num=10, variants=3, no sweep/seed) the equivalent counts would have been roughly one-third to one-quarter of this.

---

6. Verdict

- **Primary culprit:** `42e7ddf` — tripling results-per-query (`DISCOVERY_DEEP_NUM` =30 vs the old `searchDepth` =10) across the two highest-recall lanes, plus variants 3→4 and the extra analyst-PDF query. This is the single biggest multiplier of discovery time.
- **Strong secondary culprit:** `fe26685` — the issuer-domain PDF sweep (up to 25 extra PDFs per domain) and per-document aboutness scoring.
- **Compounding env setting:** `DISCOVERY_DEEP_NUM = 30` (the code default; also set in Railway). If this were 10 the lane yields would match the baseline.
- **Not the cause:** `aba322c` domain filters (1 corpus failure vs 47 timeouts disproves it) and `bb8e08b` (worker.ts only — the already-fixed stacking bug).

Both culprit commits are “more recall” features. Individually each is modest; shipped together on top of `DISCOVERY_DEEP_NUM=30` they multiplied per-company discovery work past the unchanged 20-minute ceiling.

---

7. Fix options (for decision — balances speed vs. discovery quality)

The goal is to get discovery back inside its budget **without gutting recall**. Options, smallest-blast-radius first:

1. **Lower `DISCOVERY_DEEP_NUM` 30 → 12–15** (env only, instant, reversible). Restores most of the baseline per-query cost while keeping some of the deeper-recall intent. Biggest win per unit risk.
2. **Scope the issuer PDF sweep:** reduce `ISSUER_SWEEP_MAX` 25 → ~10, or gate the sweep to the primary domain only (not every related domain).
3. **Variants 4 → 3** (`discovery_query_variants`) — small, undoes part of 4a(ii).
4. **If recall must stay at full depth, raise the budgets instead:** `DISCOVERY_TIMEOUT_MS` and the job watchdog — but this makes a 50-company single run much longer and risks PG pool starvation, so prefer trimming workload.

Recommendation: start with (1) + (2) — they target the exact multipliers proven above, are structural (not per-company patches), keep fail-on-timeout intact, and need no corpus pinning. Then run one 50-company batch to confirm. Final depth numbers are your call given you want answers “as close to perfect as possible.”